

DOCUMENTATION

Tests Unitaires

Projet Marais 2026 - Sonde TVOC/CO2

Come PETIT - Mateo ROUSSEAU

27 / 04 / 2026

1. Definition du test unitaire

Un test unitaire est une procédure permettant de vérifier le bon fonctionnement d'une portion isolée d'un programme (fonction, méthode ou classe). L'objectif est de valider que chaque unité de code produit le résultat attendu, indépendamment des autres composants.

1.1 Principes fondamentaux

- * Isolation : chaque test est indépendant et ne dépend d'aucun état externe.
- * Reproductibilité : le test produit toujours le même résultat dans les mêmes conditions.
- * Automatisation : les tests sont exécutés automatiquement via un framework (ici unittest Python).
- * Rapidité : chaque test s'exécute en quelques millisecondes.

1.2 Framework utilise : unittest

Le projet utilise le module standard Python unittest, complété par unittest.mock pour simuler les dépendances matérielles.

Composant	Rôle
unittest.TestCase	Classe de base pour écrire les cas de test
unittest.mock.MagicMock	Objet simulant n'importe quelle dépendance
unittest.mock.patch	Remplacement temporaire d'un module ou objet pendant un test
setUp()	Méthode exécutée avant chaque test pour préparer l'environnement
assertXxx()	Méthodes de vérification (assertEqual, assertTrue, assertIsNone...)

1.3 Pourquoi mocker les dépendances matérielles ?

Le projet tourne sur une carte UNIHAKER connectée à un capteur CCS811 (I2C) et à un broker MQTT. Les tests unitaires ne peuvent pas dépendre de ce matériel - ils doivent fonctionner sur n'importe quelle machine de développement.

Les mocks remplacent :

- * Le bus I2C (pinpong.board.I2C) => simule par MagicMock
- * La carte UNIHAKER (pinpong.board.Board) => patchée avec @patch

- * Le client MQTT (`paho.mqtt.client.Client`) => remplace par `MagicMock`

2. Architecture du projet

Le projet est structuré en modules Python indépendants, chacun ayant une responsabilité bien définie. L'ensemble forme une sonde de qualité d'air qui mesure le CO2 équivalent (eCO2) et les composés organiques volatils totaux (TVOC).

2.1 Diagramme de classes

Classe	Fichier	Responsabilite
Ccs811	Ccs811.py	Pilote bas niveau du capteur CCS811 (I2C, registres, mesures)
MesureTVOC_CO2	MesureTVOC_CO2.py	Gestion haut niveau : run-in, reconnexion, donnees valides
MQTT	MQTT.py	Client MQTT : connexion broker, publication JSON des mesures
Controller	Controller.py	Orchestration : boucle principale, appel capteur + publication
IHM	IHM.py	Interface utilisateur (Kivy) - non couverte par les tests actuels

2.2 Flux de donnees

- * Le Controller instancie MeasureTVOC_CO2 et MQTT.
- * MeasureTVOC_CO2 initialise la carte UNIHAKER et le capteur Ccs811.
- * Toutes les 60 secondes, get_mesures() est appelee.
- * Si une mesure valide est disponible (hors run-in, capteur connecte, donnees pretes), elle est publiee via MQTT.
- * Le topic MQTT inclut l'adresse MAC de la sonde : marais/sondes/<MAC>.
- * Le payload JSON contient un timestamp ISO 8601 et les valeurs eCO2 / TVOC.

2.3 Format du message MQTT

```
{
  "timestamp": "2026-04-27T14:32:10.123456",
  "measure": {
    "ECO2": 650,
    "TVOC": 42
  }
}
```

3. Classes a tester

3.1 Classe Ccs811

Pilote bas niveau du capteur de qualite d'air. Communique directement avec le composant via le bus I2C a l'adresse 0x5A.

Methode	Signature	Description
read_reg	read_reg(reg, length=1)	Lit length octets depuis le registre reg du CCS811
write_reg	write_reg(reg, data_bytes)	Ecrit data_bytes dans le registre reg
ccs811_init	ccs811_init()	Initialise le capteur : scan I2C, verification HW_ID, demarrage app, mode mesure 1s
data_ready	data_ready()	Retourne True si bit 3 du registre STATUS est a 1 (donnee disponible)
read_eco2_tvoc	read_eco2_tvoc()	Lit 4 octets du registre ALG_RESULT et retourne (eco2, tvoc)

Registres principaux du CCS811 :

Constante	Adresse	Role
REG_STATUS	0x00	Registre d'etat (bit 3 = DATA_READY)
REG_MEAS_MODE	0x01	Mode de mesure (0x10 = 1 mesure/seconde)
REG_ALG_RESULT	0x02	Resultat eCO2 (2 octets) + TVOC (2 octets)
REG_HW_ID	0x20	Identifiant materiel (attendu : 0x81)
REG_APP_START	0xF4	Demarrage de l'application firmware

3.2 Classe MesureTVOC_CO2

Encapsule la logique metier : gestion du run-in (30 secondes apres init), detection de deconnexion du capteur et reinitialisation automatique.

Methode	Description	Retour
__init__	Initialise les attributs et appelle _init_hardware()	—
_init_hardware	Initialise la carte UNIHAKER et le capteur Ccs811	True si succes, False sinon
get_mesures	Retourne une mesure valide ou None selon l'etat du capteur	(eco2, tvoc) ou None

3.3 Classe MQTT

Gere la connexion au broker et la publication des mesures. L'identifiant client et le topic sont generes depuis l'adresse MAC de la machine.

Methode	Description
__init__	Initialise la connexion au broker marais2026.btssn.ovh:1883
_connect	Cree le client paho, configure les credentials, lance la boucle reseau
publish_measure(eco2, tvoc)	Publie un message JSON horodate sur le topic marais/sondes/<MAC>
disconnect	Arrete la boucle reseau et deconnecte proprement le client

4. Tests unitaires

4.1 Fichier test_Ccs811.py

Classe testee : Ccs811 | Framework : unittest | Mocks : i2c remplace par MagicMock dans setUp()

test_read_reg — Lecture d'un registre

Verifie que read_reg appelle correctement i2c.readfrom_mem avec les bons parametres et retourne la liste d'octets attendue.

```
def test_read_reg(self):
    self.capteur.i2c.readfrom_mem.return_value = bytes([0x81])
    resultat = self.capteur.read_reg(self.capteur.REG_HW_ID, 1)
    self.assertEqual(resultat, [0x81])
    self.capteur.i2c.readfrom_mem.assert_called_once_with(
        self.capteur.CCS811_ADDR, self.capteur.REG_HW_ID, 1
    )
```

Entree	Resultat attendu	Resultat
REG_HW_ID (0x20), length=1	[0x81]	PASS

test_write_reg — Ecriture dans un registre

Verifie que write_reg appelle i2c.writeto_mem avec l'adresse, le registre et les donnees corrects.

```
def test_write_reg(self):
    self.capteur.write_reg(self.capteur.REG_MEAS_MODE, [0x10])
    self.capteur.i2c.writeto_mem.assert_called_once_with(
        self.capteur.CCS811_ADDR, self.capteur.REG_MEAS_MODE, [0x10]
    )
```

Entree	Resultat attendu	Resultat
REG_MEAS_MODE, [0x10]	writeto_mem appele avec les bons args	PASS

test_data_ready_true / test_data_ready_false — Etat DATA_READY

Verifie que data_ready() interprete correctement le bit 3 du registre STATUS.

```
def test_data_ready_true(self):
    self.capteur.read_reg = MagicMock(return_value=[0x08]) # bit 3 = 1
    self.assertTrue(self.capteur.data_ready())

def test_data_ready_false(self):
    self.capteur.read_reg = MagicMock(return_value=[0x00]) # bit 3 = 0
    self.assertFalse(self.capteur.data_ready())
```

Valeur STATUS	Resultat attendu	Resultat
0x08 (bit 3 = 1)	True	PASS
0x00 (bit 3 = 0)	False	PASS

test_read_eco2_tvoc — Decodage des mesures

Verifie que les 4 octets du registre ALG_RESULT sont correctement decodes en eCO2 et TVOC (2 octets big-endian chacun).

```
def test_read_eco2_tvoc(self):
    self.capteur.read_reg = MagicMock(return_value=[0x04, 0xD2, 0x00, 0x64])
    eco2, tvoc = self.capteur.read_eco2_tvoc()
    self.assertEqual(eco2, 1234) # 0x04D2 = 1234
    self.assertEqual(tvoc, 100) # 0x0064 = 100
```

Donnees brutes	eco2 attendu	tvoc attendu	Resultat
[0x04, 0xD2, 0x00, 0x64]	1234	100	PASS

test_ccs811_init_ok — Initialisation nominale

Scenario nominal : scan I2C trouve 0x5A, HW_ID est 0x81, l'application démarre et le mode mesure est configure.

```
@patch('time.sleep', return_value=None)
def test_ccs811_init_ok(self, _):
    self.capteur.i2c.scan.return_value = [0x5A]
    self.capteur.read_reg = MagicMock(return_value=[0x81])
    self.capteur.write_reg = MagicMock()
    self.capteur.ccs811_init()
    self.capteur.i2c.writeto.assert_called_once_with(
        self.capteur.CCS811_ADDR, [self.capteur.REG_APP_START]
    )
    self.capteur.write_reg.assert_called_once_with(
        self.capteur.REG_MEAS_MODE, [0x10]
    )
```

Scenario	Resultat attendu	Resultat
scan=[0x5A], HW_ID=0x81	APP_START écrit,	PASS

Scenario	Resultat attendu	Resultat
	MEAS_MODE=0x10	

test_ccs811_init_capteur_absent — Capteur non detecte

Verifie qu'une RuntimeError est levee si le scan I2C ne retourne pas l'adresse 0x5A.

```
def test_ccs811_init_capteur_absent(self):
    self.capteur.i2c.scan.return_value = [] # aucun peripherique
    with self.assertRaises(RuntimeError):
        self.capteur.ccs811_init()
```

Scenario	Exception attendue	Resultat
scan=[] (vide)	RuntimeError	PASS

test_ccs811_init_hw_id_incorrect — Mauvais identifiant materiel

Verifie qu'une RuntimeError est levee si le HW_ID lu ne vaut pas 0x81.

```
def test_ccs811_init_hw_id_incorrect(self):
    self.capteur.i2c.scan.return_value = [0x5A]
    self.capteur.read_reg = MagicMock(return_value=[0x00]) # mauvais HW_ID
    with self.assertRaises(RuntimeError):
        self.capteur.ccs811_init()
```

Scenario	Exception attendue	Resultat
HW_ID=0x00 (attendu 0x81)	RuntimeError	PASS

4.2 Fichier test_MesureTVOC_CO2.py

Classe testee : *MesureTVOC_CO2* | Mocks : *Board, Ccs811, time.time* patches via *@patch*

test_init_hardware_success — Initialisation reussie

Verifie que `_init_hardware()` retourne `True`, que `board_inited` passe a `True` et que `ccs811` n'est pas `None` apres une initialisation reussie.

```
def test_init_hardware_success(self):
    result = self.measure._init_hardware()
    self.assertTrue(result)
    self.assertTrue(self.measure.board_inited)
    self.assertIsNotNone(self.measure.ccs811)
```

Verification	Resultat
Valeur de retour == True	PASS
board_inited == True	PASS
ccs811 is not None	PASS

test_init_hardware_fail — Echec d'initialisation du capteur

Simule une exception lors de la creation du `Ccs811` pour verifier que `_init_hardware()` retourne `False` et que `ccs811` reste `None`.

```
@patch('MeasureTVOC_CO2.Board')
@patch('MeasureTVOC_CO2.Ccs811')
def test_init_hardware_fail(self, MockCcs811, MockBoard):
    MockCcs811.side_effect = Exception('Init fail')
    result = self.measure._init_hardware()
    self.assertFalse(result)
    self.assertIsNone(self.measure.ccs811)
```

Condition	Resultat attendu	Resultat
Ccs811() leve Exception	False retourne, ccs811=None	PASS

test_get_mesures_ccs811_none_init_success — Reinitialisation automatique

Verifie que si `ccs811` est `None`, `get_mesures()` tente automatiquement une reinitialisation via `_init_hardware()`. Le resultat reste `None` car le run-in n'est pas ecole.

```
def test_get_mesures_ccs811_none_init_success(self):
    self.measure.ccs811 = None
    with patch.object(self.measure, '_init_hardware', return_value=True) as mock_init:
        result = self.measure.get_mesures()
    self.assertIsNone(result)
    mock_init.assert_called_once()
```

Condition	Resultat attendu	Resultat
ccs811=None, init reussit	_init_hardware appele, None retourne	PASS

test_get_mesures_i2c_fail — Deconnexion du capteur

Verifie que si la communication I2C echoue, get_mesures() retourne None et remet ccs811 a None pour forcer une reinitialisation au prochain appel.

```
def test_get_mesures_i2c_fail(self):
    self.measure.ccs811 = Mock()
    self.measure.ccs811.i2c.readfrom_mem.side_effect = Exception()
    result = self.measure.get_mesures()
    self.assertIsNone(result)
    self.assertIsNone(self.measure.ccs811)
```

Condition	Resultat attendu	Resultat
I2C readfrom_mem leve Exception	None retourne, ccs811 mis a None	PASS

test_get_mesures_runin_period — Periode de run-in

Verifie que la methode retourne None si le delai de chauffe (30 secondes) n'est pas encore ecoule depuis la derniere initialisation.

```
@patch('time.time', return_value=1000)
def test_get_mesures_runin_period(self, mock_time):
    self.measure.ccs811 = Mock()
    self.measure.ccs811.i2c.readfrom_mem.return_value = b'\x00'
    self.measure.reinit_time = 990 # 10s < 30s => run-in actif
    result = self.measure.get_mesures()
    self.assertIsNone(result)
```

time.time()	reinit_time	Duree ecoulee	Resultat attendu	Resultat
1000	990	10s < 30s	None (run-in)	PASS

test_get_mesures_success — Mesure valide

Scenario nominal : run-in termine, donnees pretes. Verifie que get_mesures() retourne bien le tuple (eco2, tvoc).

```
@patch('time.time', return_value=1000)
def test_get_mesures_success(self, mock_time):
    self.measure.ccs811 = Mock()
    self.measure.ccs811.i2c.readfrom_mem.return_value = b'\x00'
    self.measure.reinit_time = 900 # 100s > 30s => run-in termine
    self.measure.ccs811.data_ready.return_value = True
    self.measure.ccs811.read_eco2_tvoc.return_value = (1234, 56)
    result = self.measure.get_mesures()
    self.assertEqual(result, (1234, 56))
```

Condition	Resultat attendu	Resultat
run-in OK, data_ready=True, (1234,56)	(1234, 56)	PASS

4.3 Fichier test_MQTT.py

Classe testee : MQTT | Mocks : paho.mqtt.client.Client patche via @patch('MQTT.mqtt.Client')

test_init — Creation et connexion au broker

Verifie que l'instanciation cree bien un objet et appelle client.connect() pour etabli la connexion au broker.

```
@patch('MQTT.mqtt.Client')
def test_init(self, mock_client_class):
    mock_client = MagicMock()
    mock_client_class.return_value = mock_client
    obj = MQTT()
    self.assertIsNotNone(obj)
    mock_client.connect.assert_called()
```

Verification	Resultat
obj is not None	PASS
client.connect() appele	PASS

test_publish — Publication d'un message

Verifie que l'appel a publish declenche client.publish(). Le test utilise hasattr() pour s'adapter dynamiquement au nom de la methode.

```
@patch('MQTT.mqtt.Client')
def test_publish(self, mock_client_class):
    mock_client = MagicMock()
    mock_client_class.return_value = mock_client
    obj = MQTT()
    if hasattr(obj, 'publish'):
        obj.publish('bonjour')
    mock_client.publish.assert_called()
```

Methode detectee	Verification	Resultat
publish / publier	client.publish() appele	A ameliorer : tester publish_measure

Note : La classe MQTT expose publish_measure(eco2, tvoc) et non publish(). Le test ne couvrira pas cette methode avec hasattr(). Amelioration recommandee : tester directement obj.publish_measure(400, 25) et verifier le payload JSON genere.

test_subscribe — Abonnement a un topic

Verifie que la methode d'abonnement appelle client.subscribe(). La classe MQTT actuelle ne dispose pas de cette methode.

Verification	Resultat
client.subscribe() appele	A ameliorer : methode absente de la classe actuelle

Note : La classe MQTT actuelle ne propose pas de methode subscribe(). Ce test met en evidence une fonctionnalite manquante si une communication bidirectionnelle est souhaitee.

5. Bilan des tests

5.1 Recapitulatif global

Fichier de test	Nb tests	PASS	Remarques
test_Ccs811.py	7	7 / 7	Couverture complete du pilote CCS811
test_MesureTVOC_CO2.py	6	6 / 6	Tous les scenarios (run-in, deconnexion, succes)
test_MQTT.py	3	1 / 3	test_publish et test_subscribe a ameliorer
TOTAL	16	14 / 16	

5.2 Couverture fonctionnelle

Fonctionnalite	Testee	Niveau
Lecture registre I2C	Oui	PASS
Ecriture registre I2C	Oui	PASS
Detection capteur (scan I2C)	Oui	PASS
Verification HW_ID	Oui	PASS
Decodage eCO2 / TVOC	Oui	PASS
Run-in 30 secondes	Oui	PASS
Reconnexion auto capteur	Oui	PASS
Connexion broker MQTT	Oui	PASS
Publication JSON MQTT	Partielle	A ameliorer
Abonnement MQTT	Non	A ameliorer

5.3 Recommandations

- * Ajouter un test test_publish_measure dans test_MQTT.py pour verifier le payload JSON (presence des cles timestamp, ECO2, TVOC).
- * Implementer et tester une methode subscribe() dans la classe MQTT si une communication bidirectionnelle est prevue.
- * Envisager un test d'integration Controller en mockant a la fois MesureTVOC_CO2 et MQTT.
- * Completer les tests de IHM.py une fois l'interface Kivy implementee.

